Travelling Salesman Problem (TSP)

Aim:

To find the shortest route that visits all cities exactly once and returns to the starting city.

Objective:

To minimize the total travel cost or distance while visiting every city exactly one time.

Algorithm:

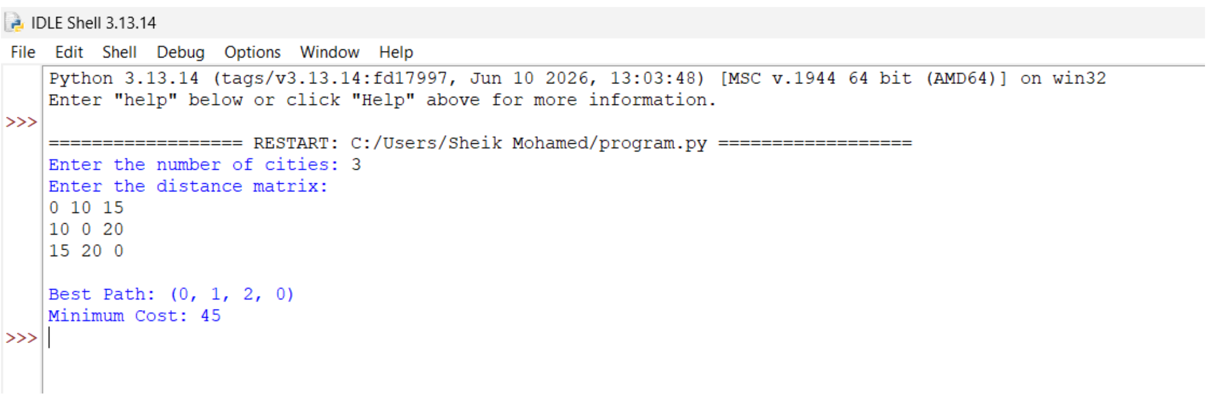
1. Start the program.
2. Input the number of cities and the distance (cost) matrix between the cities.
3. Select a city as the starting point.
4. Generate all possible routes that visit each city exactly once.
5. Calculate the total distance (cost) for each route, including the return to the starting city.
6. Compare the total cost of all possible routes.
7. Select the route with the minimum total cost.
8. Display the shortest route and its minimum travel cost.
9. Stop the program.

PYTHON CODE:

```
from itertools import permutations


n = int(input("Enter the number of cities: "))


cities = list(range(n))


print("Enter the distance matrix:")


dist = []
for i in range(n):
  row = list(map(int, input().split()))
```

```
    dist.append(row)

min_cost = float('inf')
best_path = None

for path in permutations(cities):
    cost = 0
    for i in range(len(path) - 1):
        cost += dist[path[i]][path[i + 1]]

    # Return to starting city
    cost += dist[path[-1]][path[0]]

    if cost < min_cost:
        min_cost = cost
        best_path = path

print("\nBest Path:", best_path + (best_path[0],))
print("Minimum Cost:", min_cost)
```

OUTPUT:



```
IDLE Shell 3.13.14
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
================== RESTART: C:/Users/Sheik Mohamed/program.py ==================
Enter the number of cities: 3
Enter the distance matrix:
0 10 15
10 0 20
15 20 0

Best Path: (0, 1, 2, 0)
Minimum Cost: 45
>>>
```

Result:

Shortest path identified.

Conclusion:

TSP helps optimize route planning.